PrintOnline.et v3.6 — Testing, Audit & Documentation Plan

> **Version:** 3.6.0 **Codename:** *Control* **Date:** 2026-05-05 **Status:** TESTING & AUDIT **Scope:** Full CMS CRUD interfaces for ALL product, pricing, delivery, and configuration data points — implementation complete, now entering verification phase

---

1. Executive Summary

v3.6 completes the deferred data management layer from v2. Every data point that is currently hardcoded, placeholder-only, or missing a CMS interface will be fully CRUD-capable inside the admin panel. No storefront changes — this is purely backend API + CMS UI work.

Module	Current State	v3.6 Target
Products CRUD	"Under Maintenance" page	Full create/edit/delete with all fields
Categories CRUD	"Under Maintenance" page	Full create/edit/delete with reorder
Product Options & Values	Visual-only (no CRUD)	Full CRUD on dedicated pages
Pricing Matrix	Table exists, no UI	Full CRUD + CSV bulk import
Designer Fee Tiers	Flat fee column only	Quantity-tiered pricing table + CMS
Delivery Zones	Hardcoded in `zones.ts`	DB-backed, CMS-configurable
Site Settings	None	Global config table (rush fee, thresholds)
Customer Account Mgmt	Read-only list	Full CRUD + self-service settings
Product Images	Upload exists	Full CRUD with reorder + delete
Stock/Quantity	Fields exist	CMS editable with validation

What Ships in v3.6

- **5 new migrations** — site_settings, delivery_zones, delivery_quantity_tiers, designer_fee_tiers, product schema additions
- **15 new API routes** — Full CRUD for settings, delivery zones, categories, product options/values, pricing matrix (including bulk import), designer fee tiers, customer account settings
- **7 new CMS pages** — Categories (list/new/edit), product options, pricing matrix, designer fees, site settings, delivery zones
- **12 new CMS components** — CategoryList/Form, ProductList, OptionsManager/OptionForm/ValueForm, PricingMatrixEditor/BulkImport, DesignerFeeTiers, SettingsPage, DeliveryZoneManager/Form
- **5 new hook files** — useSettings, useDeliveryZones, useDesignerFees, usePricingMatrix, useProductOptions
- **14 modified files** — Types, validations, delivery calculator refactor, existing CMS pages/hook updates
- **8 implementation phases** — DB → API → Hooks → Products/Categories UI → Options/Pricing UI → Settings UI → Account Management → Integration

---

2. Database Schema Changes

2.1 New Migration: `019_site_settings.sql`

```
CREATE TABLE site_settings (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  setting_key TEXT NOT NULL UNIQUE,
  setting_value JSONB NOT NULL,
  label TEXT NOT NULL,
  description TEXT,
  category TEXT NOT NULL DEFAULT 'general',
  -- categories: 'pricing', 'delivery', 'designer', 'general'
  data_type TEXT NOT NULL DEFAULT 'number',
  -- 'number', 'text', 'boolean', 'json'
  updated_by TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_site_settings_category ON site_settings(category);
CREATE INDEX idx_site_settings_key ON site_settings(setting_key);

ALTER TABLE site_settings ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Service role manages site_settings" ON site_settings
  FOR ALL USING (true) WITH CHECK (true);

CREATE TRIGGER update_site_settings_timestamp
  BEFORE UPDATE ON site_settings FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

Seed data:

Key	Value	Label	Category
`rush_fee_amount`	`500`	Rush Production Fee (ETB)	pricing
`free_delivery_threshold`	`5000`	Free Delivery Threshold (ETB)	delivery
`pickup_fee`	`0`	Pickup Fee (ETB)	delivery
`site_name`	`"PrintOnline.et"`	Site Name	general
`currency`	`"ETB"`	Default Currency	general

2.2 New Migration: `020_delivery_zones.sql`

```
CREATE TABLE delivery_zones (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  sub_city TEXT NOT NULL UNIQUE,
  base_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
  description TEXT,
  zone_label TEXT,
  display_order INTEGER DEFAULT 0,
```

```sql
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE delivery_quantity_tiers (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  zone_id UUID REFERENCES delivery_zones(id) ON DELETE CASCADE,
  min_quantity INTEGER NOT NULL,
  max_quantity INTEGER,
  multiplier DECIMAL(5,2) NOT NULL DEFAULT 1.0,
  label TEXT,
  display_order INTEGER DEFAULT 0,
  UNIQUE(zone_id, min_quantity)
);

CREATE INDEX idx_delivery_zones_active ON delivery_zones(is_active) WHERE is_active
= true;
CREATE INDEX idx_delivery_qty_tiers_zone ON delivery_quantity_tiers(zone_id);

ALTER TABLE delivery_zones ENABLE ROW LEVEL SECURITY;
ALTER TABLE delivery_quantity_tiers ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Public read delivery_zones" ON delivery_zones
  FOR SELECT USING (is_active = true);
CREATE POLICY "Service role manages delivery_zones" ON delivery_zones
  FOR ALL USING (true) WITH CHECK (true);

CREATE POLICY "Public read delivery_quantity_tiers" ON delivery_quantity_tiers
  FOR SELECT USING (true);
CREATE POLICY "Service role manages delivery_quantity_tiers" ON
delivery_quantity_tiers
  FOR ALL USING (true) WITH CHECK (true);

CREATE TRIGGER update_delivery_zones_timestamp
  BEFORE UPDATE ON delivery_zones FOR EACH ROW EXECUTE FUNCTION update_updated_at();
```

2.3 New Migration: `021_designer_fee_tiers.sql`

```sql
CREATE TABLE designer_fee_tiers (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  min_quantity INTEGER NOT NULL,
  max_quantity INTEGER,
  fee_amount DECIMAL(12,2) NOT NULL,
  label TEXT,
  display_order INTEGER DEFAULT 0,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(product_id, min_quantity)
);

CREATE INDEX idx_designer_fee_tiers_product ON designer_fee_tiers(product_id);
```

```sql
CREATE INDEX idx_designer_fee_tiers_active ON designer_fee_tiers(is_active) WHERE
is_active = true;

ALTER TABLE designer_fee_tiers ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Public read designer_fee_tiers" ON designer_fee_tiers
  FOR SELECT USING (is_active = true);
CREATE POLICY "Service role manages designer_fee_tiers" ON designer_fee_tiers
  FOR ALL USING (true) WITH CHECK (true);
```

2.4 Modified Migration: `022_product_schema_additions.sql`

```sql
ALTER TABLE products ADD COLUMN IF NOT EXISTS overview TEXT;
ALTER TABLE products ADD COLUMN IF NOT EXISTS rush_eligible BOOLEAN DEFAULT true;
ALTER TABLE products ADD COLUMN IF NOT EXISTS quantity_thresholds JSONB DEFAULT
'[]'::jsonb;
-- e.g. [{"min": 50, "max": 99, "discount": 0}, {"min": 100, "max": 499, "discount":
5}, {"min": 500, "max": null, "discount": 10}]

COMMENT ON COLUMN products.overview IS 'Rich text product overview shown on detail
page';
COMMENT ON COLUMN products.rush_eligible IS 'Whether this product supports rush
production';
COMMENT ON COLUMN products.quantity_thresholds IS 'Quantity-based pricing tiers as
JSON array';
```

2.5 New Migration: `023_rls_admin_policies.sql`

Add admin write policies for all new tables (since service_role bypasses RLS, but we need policies for the admin client if using user JWT):

```sql
-- Admin policies for categories (write access)
CREATE POLICY "Admin full access categories" ON categories FOR ALL
  USING (true) WITH CHECK (true);

-- Admin policies for products (write access)
CREATE POLICY "Admin full access products" ON products FOR ALL
  USING (true) WITH CHECK (true);

-- Admin policies for product_options
CREATE POLICY "Admin full access product_options" ON product_options FOR ALL
  USING (true) WITH CHECK (true);

-- Admin policies for product_option_values
CREATE POLICY "Admin full access product_option_values" ON product_option_values FOR
ALL
  USING (true) WITH CHECK (true);

-- Admin policies for product_images
CREATE POLICY "Admin full access product_images" ON product_images FOR ALL
```

```
    USING (true) WITH CHECK (true);

  -- Admin policies for pricing_matrix
  CREATE POLICY "Admin full access pricing_matrix" ON product_pricing_matrix FOR ALL
    USING (true) WITH CHECK (true);
```

2.6 Migration Summary

Migration	Tables	Action
`019_site_settings.sql`	`site_settings`	CREATE
`020_delivery_zones.sql`	`delivery_zones`, `delivery_quantity_tiers`	CREATE
`021_designer_fee_tiers.sql`	`designer_fee_tiers`	CREATE
`022_product_schema_additions.sql`	`products` (ALTER)	ALTER
`023_rls_admin_policies.sql`	All tables	ADD POLICIES

---

3. API Routes

3.1 New API Routes

Route	Methods	Purpose
`app/api/cms/settings/route.ts`	GET, PUT	Site settings CRUD
`app/api/cms/delivery-zones/route.ts`	GET, POST, PUT, DELETE	Delivery zones CRUD
`app/api/cms/delivery-zones/[id]/route.ts`	GET, PUT, DELETE	Single zone CRUD
`app/api/cms/categories/route.ts`	GET, POST, PUT, DELETE	Categories CRUD
`app/api/cms/categories/[id]/route.ts`	GET, PUT, DELETE	Single category CRUD
`app/api/cms/products/[id]/route.ts`	GET, PUT, DELETE	Single product CRUD

Route	Methods	Purpose
`app/api/cms/products/[id]/options/route.ts`	GET, POST	Product options CRUD
`app/api/cms/products/[id]/options/[optionId]/route.ts`	PUT, DELETE	Single option CRUD
`app/api/cms/products/[id]/options/[optionId]/values/route.ts`	GET, POST	Option values CRUD
`app/api/cms/products/[id]/options/[optionId]/values/[valueId]/route.ts`	PUT, DELETE	Single value CRUD
`app/api/cms/products/[id]/pricing-matrix/route.ts`	GET, POST, DELETE	Pricing matrix CRUD
`app/api/cms/products/[id]/pricing-matrix/bulk/route.ts`	POST	Bulk import pricing matrix
`app/api/cms/products/[id]/designer-fees/route.ts`	GET, POST	Designer fee tiers CRUD
`app/api/cms/products/[id]/designer-fees/[tierId]/route.ts`	PUT, DELETE	Single tier CRUD
`app/api/account/settings/route.ts`	GET, PUT	Customer self-service settings
`app/api/account/password/route.ts`	PUT	Customer password change

3.2 Modified API Routes

Route	Change
`app/api/cms/products/route.ts`	Add POST/PUT for new fields (overview, rush_eligible, quantity_thresholds)
`app/api/orders/route.ts`	Read rush_fee and delivery settings from DB instead of hardcoded values

3.3 API Pattern

All CMS API routes follow the existing pattern:

1. `auth.api.getSession()` for authentication
2. `isAdmin()` check for authorization
3. Zod schema validation on inputs
4. `supabaseAdmin` for database operations
5. Structured JSON responses `{ success, data/error }`

---

4. CMS Pages & Components

4.1 New CMS Pages

Page	Route	Purpose
Category List	`app/(cms)/cms/categories/page.tsx`	Replace "Under Maintenance" with functional list
Category Edit	`app/(cms)/cms/categories/[id]/page.tsx`	Edit category details
Category New	`app/(cms)/cms/categories/new/page.tsx`	Create new category
Product Options	`app/(cms)/cms/products/[id]/options/page.tsx`	Dedicated options management page
Product Option Edit	`app/(cms)/cms/products/[id]/options/[optionId]/page.tsx`	Edit single option + its values
Product Pricing Matrix	`app/(cms)/cms/products/[id]/pricing/page.tsx`	Pricing matrix CRUD + bulk import
Product Designer Fees	`app/(cms)/cms/products/[id]/designer-fees/page.tsx`	Designer fee tiers management
Site Settings	`app/(cms)/cms/settings/page.tsx`	Global site settings
Delivery Zones	`app/(cms)/cms/settings/delivery/page.tsx`	Delivery zone management
Customer Account Settings	`app/(account)/account/settings/page.tsx`	Self-service account settings

4.2 Modified CMS Pages

Page	Change
`app/(cms)/cms/products/page.tsx`	Replace "Under Maintenance" with functional product list

Page	Change
`app/(cms)/cms/products/[id]/page.tsx`	Add tabs/links for options, pricing, designer fees, images
`app/(cms)/cms/products/[id]/edit/page.tsx`	Add all new product fields to edit form
`app/(cms)/cms/products/new/page.tsx`	Add all new product fields to create form
`components/cms/layout/CMSSidebar.tsx`	Remove maintenance badges, add Settings nav

4.3 New Components

Component	Purpose
`components/cms/categories/CategoryList.tsx`	Data table with search, active/inactive toggle
`components/cms/categories/CategoryForm.tsx`	Create/edit form (name, slug, description, image, SEO)
`components/cms/products/ProductList.tsx`	Functional product list with filters
`components/cms/products/OptionsManager.tsx`	Full CRUD for product options + values
`components/cms/products/OptionForm.tsx`	Add/edit option modal/sheet
`components/cms/products/OptionValueForm.tsx`	Add/edit option value modal/sheet
`components/cms/products/PricingMatrixEditor.tsx`	Row CRUD table for pricing matrix
`components/cms/products/PricingMatrixBulkImport.tsx`	CSV/paste bulk import modal
`components/cms/products/DesignerFeeTiers.tsx`	Tiered fee CRUD table
`components/cms/products/ProductFormFields.tsx`	Extended form fields (overview, rush, thresholds)
`components/cms/settings/SettingsPage.tsx`	Site settings form
`components/cms/settings/DeliveryZoneManager.tsx`	Delivery zone CRUD table
`components/cms/settings/DeliveryZoneForm.tsx`	Add/edit delivery zone form
`components/cms/shared/CMSBulkImport.tsx`	Reusable bulk import component

4.4 Sidebar Navigation Update

```
// CMSSidebar.tsx — Updated navigation (remove maintenance badges)
const navigation = [
  { href: "/cms", icon: BarChart3, label: "Overview" },
  { href: "/cms/orders", icon: ShoppingCart, label: "Orders" },
  { href: "/cms/messages", icon: MessageSquare, label: "Messages" },
  { href: "/cms/products", icon: Package, label: "Products" },          // ← Remove
maintenance
  { href: "/cms/customers", icon: Users, label: "Customers" },
  { href: "/cms/team", icon: ShieldCheck, label: "Team" },
```

```
  { href: "/cms/categories", icon: Layers, label: "Categories" },       // ← Remove
maintenance
  { href: "/cms/settings", icon: Settings, label: "Settings" },         // ← NEW
];
```

---

5. Data Hooks (TanStack Query)

5.1 New Hooks

Hook	File	Purpose
`useSiteSettings`	`hooks/data/useSettings.ts`	Fetch all site settings
`useUpdateSiteSetting`	`hooks/data/useSettings.ts`	Mutation to update a setting
`useDeliveryZones`	`hooks/data/useDeliveryZones.ts`	Fetch all delivery zones
`useCreateDeliveryZone`	`hooks/data/useDeliveryZones.ts`	Mutation to create zone
`useUpdateDeliveryZone`	`hooks/data/useDeliveryZones.ts`	Mutation to update zone
`useDeleteDeliveryZone`	`hooks/data/useDeliveryZones.ts`	Mutation to delete zone
`useDesignerFeeTiers`	`hooks/data/useDesignerFees.ts`	Fetch tiers for a product
`useCreateDesignerFeeTier`	`hooks/data/useDesignerFees.ts`	Mutation to create tier
`useUpdateDesignerFeeTier`	`hooks/data/useDesignerFees.ts`	Mutation to update tier
`useDeleteDesignerFeeTier`	`hooks/data/useDesignerFees.ts`	Mutation to delete tier
`usePricingMatrix`	`hooks/data/usePricingMatrix.ts`	Fetch matrix for a product
`useCreatePricingEntry`	`hooks/data/usePricingMatrix.ts`	Mutation to create entry
`useUpdatePricingEntry`	`hooks/data/usePricingMatrix.ts`	Mutation to update entry
`useDeletePricingEntry`	`hooks/data/usePricingMatrix.ts`	Mutation to delete entry
`useBulkImportPricing`	`hooks/data/usePricingMatrix.ts`	Mutation for CSV bulk import
`useProductOptions`	`hooks/data/useProductOptions.ts`	Fetch options for a product
`useCreateOption`	`hooks/data/useProductOptions.ts`	Mutation to create option
`useUpdateOption`	`hooks/data/useProductOptions.ts`	Mutation to update option
`useDeleteOption`	`hooks/data/useProductOptions.ts`	Mutation to delete option
`useCreateOptionValue`	`hooks/data/useProductOptions.ts`	Mutation to create value
`useUpdateOptionValue`	`hooks/data/useProductOptions.ts`	Mutation to update value
`useDeleteOptionValue`	`hooks/data/useProductOptions.ts`	Mutation to delete value
`useCreateCategory`	`hooks/data/useCategories.ts`	Mutation to create category
`useUpdateCategory`	`hooks/data/useCategories.ts`	Mutation to update category

Hook	File	Purpose
`useDeleteCategory`	`hooks/data/useCategories.ts`	Mutation to delete category
`useCreateProduct`	`hooks/data/useProducts.ts`	Mutation to create product
`useUpdateProduct`	`hooks/data/useProducts.ts`	Mutation to update product
`useDeleteProduct`	`hooks/data/useProducts.ts`	Mutation to delete product

5.2 Modified Hooks

Hook	Change
`hooks/data/useCategories.ts`	Add `useAllCategories()` (include inactive) for CMS
`hooks/data/useProducts.ts`	Add `useAllProducts()` (include inactive) for CMS

---

6. Types Updates

6.1 Database Types (`types/database.ts`)

Add to `Database["public"]["Tables"]`:

- `site_settings` — Row/Insert/Update types
- `delivery_zones` — Row/Insert/Update types
- `delivery_quantity_tiers` — Row/Insert/Update types
- `designer_fee_tiers` — Row/Insert/Update types

Update `products` Row type to include:

- `overview: string | null`
- `rush_eligible: boolean | null`
- `quantity_thresholds: Json | null`

6.2 Convenience Type Aliases

```
export type SiteSetting = Database["public"]["Tables"]["site_settings"]["Row"];
export type DeliveryZone = Database["public"]["Tables"]["delivery_zones"]["Row"];
export type DeliveryQuantityTier = Database["public"]["Tables"]
["delivery_quantity_tiers"]["Row"];
export type DesignerFeeTier = Database["public"]["Tables"]["designer_fee_tiers"]
["Row"];
```

6.3 Domain Types (`types/index.ts`)

```
export type ProductWithFullDetails = Product & {
  category: Pick<Category, "name" | "slug"> | null;
  product_images: ProductImage[];
  product_options: (ProductOption & {
    product_option_values: ProductOptionValue[];
```

```
  })[];
  pricing_matrix: PricingMatrixEntry[];
  designer_fee_tiers: DesignerFeeTier[];
};
```

---

7. Validation Schemas

7.1 New Schemas (`lib/validations/cms.ts`)

```
// Site Settings
export const siteSettingSchema = z.object({
  setting_key: z.string().min(1),
  setting_value: z.any(),
  label: z.string().min(1),
  description: z.string().optional(),
  category: z.enum(["pricing", "delivery", "designer", "general"]),
  data_type: z.enum(["number", "text", "boolean", "json"]),
});

// Delivery Zone
export const deliveryZoneSchema = z.object({
  sub_city: z.string().min(1),
  base_fee: z.number().min(0),
  description: z.string().optional(),
  zone_label: z.string().optional(),
  display_order: z.number().int().min(0).default(0),
  is_active: z.boolean().default(true),
});

// Designer Fee Tier
export const designerFeeTierSchema = z.object({
  product_id: z.string().uuid(),
  min_quantity: z.number().int().min(1),
  max_quantity: z.number().int().min(1).optional().nullable(),
  fee_amount: z.number().min(0),
  label: z.string().optional(),
  display_order: z.number().int().min(0).default(0),
  is_active: z.boolean().default(true),
});

// Pricing Matrix Entry
export const pricingMatrixEntrySchema = z.object({
  product_id: z.string().uuid(),
  matrix_key: z.string().min(1),
  matrix_label: z.string().optional(),
  price: z.number().min(0),
  is_active: z.boolean().default(true),
});

// Bulk Import
export const pricingMatrixBulkImportSchema = z.object({
  product_id: z.string().uuid(),
```

```
  entries: z.array(pricingMatrixEntrySchema.omit({ product_id: true })),
});
```

---

8. Delivery System Refactor

8.1 Refactor `lib/delivery/zones.ts`

Replace hardcoded data with DB-backed fetcher:

```
// Before: Hardcoded DELIVERY_ZONES array
// After: Fetch from delivery_zones table
import { createClient } from "@/lib/supabase/client";

export async function fetchDeliveryZones(): Promise<DeliveryZone[]> {
  const supabase = createClient();
  const { data, error } = await supabase
    .from("delivery_zones")
    .select("*")
    .eq("is_active", true)
    .order("display_order");
  if (error) throw error;
  return data ?? [];
}

export async function fetchFreeDeliveryThreshold(): Promise<number> {
  const supabase = createClient();
  const { data } = await supabase
    .from("site_settings")
    .select("setting_value")
    .eq("setting_key", "free_delivery_threshold")
    .single();
  return data ? Number(data.setting_value) : 5000;
}
```

8.2 Refactor `lib/delivery/calculator.ts`

Update to use async DB lookups. The `calculateDeliveryFee` function becomes async and reads from the database instead of hardcoded constants.

8.3 Rush Fee Refactor

Currently hardcoded at 500 in `components/product/ProductOrderForm.tsx`. After v3.6:

- Read from `site_settings` table (`rush_fee_amount` key)
- Pass as prop or fetch via hook in the storefront product form

---

9. Implementation Phases

Phase 1: Database Layer (Migrations + Types)

1. Create migration files 019-023
2. Create seed data for site_settings and delivery_zones
3. Update `types/database.ts` with new table types
4. Update convenience type aliases

Phase 2: API Layer

1. Create all new API routes (settings, delivery zones, categories, options, pricing, designer fees, images)
2. Update existing API routes (products, orders)
3. Add Zod validation schemas

Phase 3: Hooks Layer

1. Create all new TanStack Query hooks
2. Update existing hooks with mutations

Phase 4: CMS UI — Products & Categories

1. Replace "Under Maintenance" pages with functional lists
2. Build CategoryForm + CategoryList components
3. Build functional ProductList component
4. Update product create/edit forms with new fields

Phase 5: CMS UI — Options & Pricing

1. Build dedicated Options management page
2. Build OptionForm + OptionValueForm components
3. Build PricingMatrixEditor with row CRUD
4. Build PricingMatrixBulkImport (CSV/paste)
5. Build DesignerFeeTiers component

Phase 6: CMS UI — Settings & Delivery

1. Build Site Settings page
2. Build DeliveryZoneManager + DeliveryZoneForm
3. Add Settings nav to sidebar

Phase 7: Customer Account Management

1. Build customer CRUD in CMS (edit/deactivate profiles)
2. Build customer self-service settings page (`/account/settings`)
3. Build password change API + UI

Phase 8: Integration & Cleanup

1. Refactor delivery calculator to use DB
2. Refactor rush fee to use site_settings
3. Remove maintenance badges from sidebar
4. Update all product-related hooks to include new fields
5. End-to-end testing of all CRUD flows

---

10. Execution Order & Dependencies

Phases 4-7 can be worked on in parallel once Phase 3 is complete.

```
Phase 1 (DB)  ── no deps ──────────────────────┐
                                               │
Phase 2 (API) ── depends on Phase 1 ───────────┤
                                               │
Phase 3 (Hooks) ── depends on Phase 2 ─────────┤
                                               │
Phase 4 (Products/Categories UI) ── depends on Phase 3
Phase 5 (Options/Pricing UI) ────── depends on Phase 3
Phase 6 (Settings/Delivery UI) ─── depends on Phase 3
Phase 7 (Account Management) ────── depends on Phase 3
                                               │
Phase 8 (Integration) ─── depends on all above ─┘
```

---

11. Complete File Manifest

New Files (44 files)

Migrations (5):

- `supabase/migrations/019_site_settings.sql`
- `supabase/migrations/020_delivery_zones.sql`
- `supabase/migrations/021_designer_fee_tiers.sql`
- `supabase/migrations/022_product_schema_additions.sql`
- `supabase/migrations/023_rls_admin_policies.sql`

Seed (1):

- `supabase/seed/014_site_settings_and_delivery_zones.sql`

API Routes (15):

- `app/api/cms/settings/route.ts`
- `app/api/cms/delivery-zones/route.ts`
- `app/api/cms/delivery-zones/[id]/route.ts`
- `app/api/cms/categories/route.ts`
- `app/api/cms/categories/[id]/route.ts`
- `app/api/cms/products/[id]/route.ts`
- `app/api/cms/products/[id]/options/route.ts`
- `app/api/cms/products/[id]/options/[optionId]/route.ts`
- `app/api/cms/products/[id]/options/[optionId]/values/route.ts`
- `app/api/cms/products/[id]/options/[optionId]/values/[valueId]/route.ts`
- `app/api/cms/products/[id]/pricing-matrix/route.ts`
- `app/api/cms/products/[id]/pricing-matrix/bulk/route.ts`
- `app/api/cms/products/[id]/designer-fees/route.ts`
- `app/api/cms/products/[id]/designer-fees/[tierId]/route.ts`

- `app/api/account/settings/route.ts`
- `app/api/account/password/route.ts`

CMS Pages (7):

- `app/(cms)/cms/categories/page.tsx`
- `app/(cms)/cms/categories/[id]/page.tsx`
- `app/(cms)/cms/categories/new/page.tsx`
- `app/(cms)/cms/products/[id]/options/page.tsx`
- `app/(cms)/cms/products/[id]/pricing/page.tsx`
- `app/(cms)/cms/products/[id]/designer-fees/page.tsx`
- `app/(cms)/cms/settings/page.tsx`
- `app/(cms)/cms/settings/delivery/page.tsx`
- `app/(account)/account/settings/page.tsx`

CMS Components (12):

- `components/cms/categories/CategoryList.tsx`
- `components/cms/categories/CategoryForm.tsx`
- `components/cms/products/ProductList.tsx`
- `components/cms/products/OptionsManager.tsx`
- `components/cms/products/OptionForm.tsx`
- `components/cms/products/OptionValueForm.tsx`
- `components/cms/products/PricingMatrixEditor.tsx`
- `components/cms/products/PricingMatrixBulkImport.tsx`
- `components/cms/products/DesignerFeeTiers.tsx`
- `components/cms/settings/SettingsPage.tsx`
- `components/cms/settings/DeliveryZoneManager.tsx`
- `components/cms/settings/DeliveryZoneForm.tsx`

Hooks (5):

- `hooks/data/useSettings.ts`
- `hooks/data/useDeliveryZones.ts`
- `hooks/data/useDesignerFees.ts`
- `hooks/data/usePricingMatrix.ts`
- `hooks/data/useProductOptions.ts`

Modified Files (14)

- `types/database.ts` — Add new table types
- `types/index.ts` — Add convenience aliases
- `lib/validations/cms.ts` — Add new Zod schemas
- `lib/delivery/calculator.ts` — Refactor to async DB lookups
- `lib/delivery/zones.ts` — Refactor to DB-backed
- `app/api/cms/products/route.ts` — Add new fields to schema
- `hooks/data/useProducts.ts` — Add mutations + useAllProducts
- `hooks/data/useCategories.ts` — Add mutations + useAllCategories
- `app/(cms)/cms/products/page.tsx` — Replace maintenance page
- `app/(cms)/cms/categories/page.tsx` — Replace maintenance page

- `components/cms/layout/CMSSidebar.tsx` — Remove maintenance badges, add Settings
- `components/cms/products/ProductForm.tsx` — Add new fields
- `components/cms/products/ProductOptionEditor.tsx` — Link to dedicated pages
- `app/(cms)/cms/products/[id]/page.tsx` — Add navigation tabs

---

12. Risk Mitigation

Risk	Mitigation
Breaking existing product display	All new fields are additive (nullable/optional). Existing queries unchanged.
Delivery calculator refactor breaks checkout	Keep hardcoded fallbacks. DB values override defaults when available.
Pricing matrix bulk import data corruption	Validate all entries server-side before insert. Use transactions.
RLS policy conflicts	All new policies use service_role pattern consistent with existing code.
Type generation drift	Manually append convenience types after regeneration. Documented in types file.

---

13. Out of Scope (v3.6)

- Mobile responsive fixes
- Dark mode fixes
- Product image upload from Supabase Storage (existing upload flow preserved)
- Search functionality changes
- Order status pipeline changes
- Email template changes
- Payment gateway changes
- Storefront UI changes (no product page redesign)

These are deferred to v3.7+ as specified in the v3 master plan.

---

14. v3.6 Implementation Audit (File Verification)

All implementation phases (1–8) are **COMPLETE**. The following files were verified to exist:

Migrations (5/5) ✅

- `supabase/migrations/019_site_settings.sql` ✅
- `supabase/migrations/020_delivery_zones.sql` ✅
- `supabase/migrations/021_designer_fee_tiers.sql` ✅
- `supabase/migrations/022_product_schema_additions.sql` ✅
- `supabase/migrations/023_rls_admin_policies.sql` ✅

Seed (1/1) ✅

- `supabase/seed/014_site_settings_and_delivery_zones.sql` ✅

API Routes (16/16) ✅

- `app/api/cms/settings/route.ts` ✅
- `app/api/cms/delivery-zones/route.ts` ✅
- `app/api/cms/delivery-zones/[id]/route.ts` ✅
- `app/api/cms/categories/route.ts` ✅
- `app/api/cms/categories/[id]/route.ts` ✅
- `app/api/cms/products/[id]/options/route.ts` ✅
- `app/api/cms/products/[id]/options/[optionId]/route.ts` ✅
- `app/api/cms/products/[id]/options/[optionId]/values/route.ts` ✅
- `app/api/cms/products/[id]/options/[optionId]/values/[valueId]/route.ts` ✅
- `app/api/cms/products/[id]/pricing-matrix/route.ts` ✅
- `app/api/cms/products/[id]/pricing-matrix/bulk/route.ts` ✅
- `app/api/cms/products/[id]/designer-fees/route.ts` ✅
- `app/api/cms/products/[id]/designer-fees/[tierId]/route.ts` ✅
- `app/api/account/settings/route.ts` ✅
- `app/api/account/password/route.ts` ✅

Hooks (5/5) ✅

- `hooks/data/useSettings.ts` ✅
- `hooks/data/useDeliveryZones.ts` ✅
- `hooks/data/useDesignerFees.ts` ✅
- `hooks/data/usePricingMatrix.ts` ✅
- `hooks/data/useProductOptions.ts` ✅

CMS Components (12/12) ✅

- `components/cms/categories/CategoryList.tsx` ✅
- `components/cms/categories/CategoryForm.tsx` ✅
- `components/cms/products/ProductList.tsx` ✅
- `components/cms/products/OptionsManager.tsx` ✅
- `components/cms/products/OptionForm.tsx` ✅
- `components/cms/products/OptionValueForm.tsx` ✅
- `components/cms/products/PricingMatrixEditor.tsx` ✅
- `components/cms/products/PricingMatrixBulkImport.tsx` ✅
- `components/cms/products/DesignerFeeTiers.tsx` ✅
- `components/cms/settings/SettingsPage.tsx` ✅
- `components/cms/settings/DeliveryZoneManager.tsx` ✅
- `components/cms/settings/DeliveryZoneForm.tsx` ✅

CMS Pages (9/9) ✅

- `app/(cms)/cms/categories/page.tsx` ✅
- `app/(cms)/cms/categories/new/page.tsx` ✅
- `app/(cms)/cms/categories/[id]/page.tsx` ✅
- `app/(cms)/cms/products/[id]/options/page.tsx` ✅
- `app/(cms)/cms/products/[id]/pricing/page.tsx` ✅

- `app/(cms)/cms/products/[id]/designer-fees/page.tsx` ✅
- `app/(cms)/cms/settings/page.tsx` ✅
- `app/(cms)/cms/settings/delivery/page.tsx` ✅
- `app/(account)/account/settings/page.tsx` ✅

UI Components (1/1) ✅

- `components/ui/switch.tsx` ✅ (newly created, `@radix-ui/react-switch` installed)

Modified Files (verified) ✅

- `types/database.ts` — 5 new table types + product columns added ✅
- `lib/validations/cms.ts` — 7 new Zod schemas added ✅
- `lib/delivery/calculator.ts` — Async DB-backed functions added ✅
- `hooks/data/useProducts.ts` — Mutations + useAllProducts added ✅
- `hooks/data/useCategories.ts` — Mutations + useAllCategories added ✅
- `hooks/data/index.ts` — Barrel exports updated ✅
- `app/(cms)/cms/products/page.tsx` — Replaced maintenance page ✅
- `app/(cms)/cms/categories/page.tsx` — Replaced maintenance page ✅
- `components/cms/layout/CMSSidebar.tsx` — Maintenance badges removed, Settings added ✅
- `components/cms/products/ProductForm.tsx` — overview + rush_eligible fields added ✅
- `app/(cms)/cms/products/[id]/page.tsx` — Options/Pricing/Fees nav buttons added ✅
- `app/(account)/layout.tsx` — Settings nav item added ✅

Build Status

- `pnpm build` — **PASS** (verified before plan mode entry)

---

15. Migration Execution Instructions (USER TASK)

> ⚠️ **You must run these 5 SQL scripts in the Supabase SQL Editor BEFORE starting manual testing.**

Execution Order

Run these in the Supabase Dashboard (SQL Editor) **in this exact order**:

#	Migration File	What It Does	Safe to Re-run?
1	`019_site_settings.sql`	Creates `site_settings` table + indexes + RLS + trigger	✅ Uses `CREATE TABLE` (will fail if exists — run once)
2	`020_delivery_zones.sql`	Creates `delivery_zones` + `delivery_quantity_tiers` tables + indexes + RLS + triggers	✅ Same
3	`021_designer_fee_tiers.sql`	Creates `designer_fee_tiers` table + indexes + RLS	✅ Same
4	`022_product_schema_additions.sql`	Adds `overview`, `rush_eligible`, `quantity_thresholds` columns to	✅ Uses `ADD COLUMN IF NOT`

#	Migration File	What It Does	Safe to Re-run?
		`products`	`EXISTS`
5	`023_rls_admin_policies.sql`	Adds admin RLS policies for categories, products, options, values, images, pricing_matrix	✅ Uses `IF NOT EXISTS` checks

Seed Data

After all 5 migrations, run the seed file:

#	Seed File	What It Does
6	`014_site_settings_and_delivery_zones.sql`	Populates `site_settings` with 5 default config rows + `delivery_zones` with 10 Addis Ababa sub-cities

Verification Queries

After running all migrations + seed, execute these verification queries:

```sql
-- 1. Verify site_settings table and seed data
SELECT setting_key, setting_value, label, category FROM site_settings ORDER BY
category, setting_key;
-- Expected: 5 rows (rush_fee_amount, free_delivery_threshold, pickup_fee,
site_name, currency)

-- 2. Verify delivery_zones table and seed data
SELECT sub_city, base_fee, zone_label, is_active FROM delivery_zones ORDER BY
display_order;
-- Expected: 10 rows (Bole=200, Kirkos=240, Arada=240, ... Lemi Kura=400)

-- 3. Verify delivery_quantity_tiers table exists
SELECT COUNT(*) FROM delivery_quantity_tiers;
-- Expected: 0 (no seed data for tiers yet — will be populated via CMS)

-- 4. Verify designer_fee_tiers table exists
SELECT COUNT(*) FROM designer_fee_tiers;
-- Expected: 0 (no seed data — will be populated via CMS)

-- 5. Verify products table has new columns
SELECT column_name, data_type, column_default
FROM information_schema.columns
WHERE table_name = 'products'
AND column_name IN ('overview', 'rush_eligible', 'quantity_thresholds')
ORDER BY column_name;
-- Expected: 3 rows (overview TEXT, rush_eligible BOOLEAN DEFAULT true,
quantity_thresholds JSONB)

-- 6. Verify RLS policies exist for new tables
SELECT tablename, policyname
FROM pg_policies
WHERE tablename IN ('site_settings', 'delivery_zones', 'delivery_quantity_tiers',
```

```
  'designer_fee_tiers')
  ORDER BY tablename, policyname;
  -- Expected: Multiple policies per table (public read + service role manage)

  -- 7. Verify triggers exist
  SELECT trigger_name, event_object_table
  FROM information_schema.triggers
  WHERE event_object_table IN ('site_settings', 'delivery_zones')
  ORDER BY event_object_table;
  -- Expected: update_site_settings_timestamp, update_delivery_zones_timestamp
```

Troubleshooting

Issue	Fix
`relation "site_settings" already exists`	Migration already ran — skip to next
`function update_updated_at() does not exist`	Run `008_functions.sql` first (should already exist from v2)
`permission denied for table site_settings`	Ensure you're using the SQL Editor with service_role or postgres role
Seed `ON CONFLICT` errors	Data already exists — safe to ignore

---

16. Automated Code Verification (I EXECUTE)

After migrations are applied, I will run these checks:

#	Check	Command/Method	Pass Criteria
1	**Build**	`pnpm build`	Exit 0, all pages generated
2	**Lint**	`pnpm lint` (120s timeout)	No new errors beyond baseline
3	**Type check**	`npx tsc --noEmit`	No new type errors beyond baseline
4	**Migration files SQL syntax**	Read + verify each migration file	Valid PostgreSQL, no syntax errors
5	**API route auth guards**	Grep all new API routes for `isAdmin` check	Every CMS route has auth + admin check
6	**Hook query keys**	Verify unique query keys across all hooks	No duplicate keys
7	**Import paths**	Verify all new files use `@/` aliases	No relative imports crossing boundaries
8	**Console.log audit**	Grep new files for `console.log`	Only in error handlers (acceptable)
9	**Zod schema coverage**	Verify every API route validates input	No unvalidated POST/PUT handlers

---

17. Manual Dev Server Testing Checklist (USER EXECUTES)

Prerequisites: `pnpm dev` running, all 5 migrations + seed applied.

T1. CMS Navigation & Sidebar (3 min)

- ☐ CMS sidebar shows "Products" WITHOUT maintenance badge
- ☐ CMS sidebar shows "Categories" WITHOUT maintenance badge
- ☐ CMS sidebar shows "Settings" link (new)
- ☐ Click "Settings" → loads `/cms/settings` page
- ☐ Click "Categories" → loads `/cms/categories` page (not maintenance placeholder)
- ☐ Click "Products" → loads `/cms/products` page (not maintenance placeholder)

T2. Categories CRUD (5 min)

- ☐ `/cms/categories` — Category list loads with existing categories
- ☐ Click "New Category" → form loads at `/cms/categories/new`
- ☐ Fill name, slug, description → Submit → category created
- ☐ Click edit on a category → form loads at `/cms/categories/[id]`
- ☐ Modify name → Save → changes persist
- ☐ Delete a category → confirm dialog → category removed

T3. Products CRUD (5 min)

- ☐ `/cms/products` — Product list loads with existing products
- ☐ Click "New Product" → form loads at `/cms/products/new`
- ☐ Form has new fields: Overview (textarea), Rush Eligible (switch)
- ☐ Fill required fields → Submit → product created
- ☐ Click on a product → detail page loads
- ☐ Detail page has "Options", "Pricing", "Fees" navigation buttons
- ☐ Click "Edit Metadata" → form loads with all fields including new ones

T4. Product Options CRUD (5 min)

- ☐ Product detail → Click "Options" → options page loads
- ☐ Click "Add Option Group" → dialog opens
- ☐ Fill label, key, field type → Create → option appears in list
- ☐ Click "Add Value" on an option → value dialog opens
- ☐ Fill label, value, price → Create → value appears as chip/tag
- ☐ Edit an option → dialog opens with pre-filled data → Save
- ☐ Delete an option → confirm dialog → option removed

T5. Pricing Matrix CRUD (5 min)

- ☐ Product detail → Click "Pricing" → pricing page loads
- ☐ Click "Add Entry" → inline row appears
- ☐ Fill key, label, price → Save → entry appears in table
- ☐ Click "Bulk Import" → modal opens
- ☐ Paste CSV data (e.g., `key1,Label One,350`) → preview shows parsed entries
- ☐ Click "Import" → entries created

- ☐ Click "Clear All" → confirm dialog → all entries deleted

T6. Designer Fee Tiers (5 min)

- ☐ Product detail → Click "Fees" → designer fees page loads
- ☐ Click "Add Tier" → inline form appears
- ☐ Fill min qty, max qty, fee amount → Save → tier appears in table
- ☐ Edit a tier → form pre-fills → Save
- ☐ Delete a tier → confirm dialog → tier removed

T7. Site Settings (5 min)

- ☐ `/cms/settings` → Settings page loads
- ☐ Settings grouped by category (Pricing, Delivery, General)
- ☐ Rush Fee Amount shows 500 ETB (from seed)
- ☐ Change Rush Fee to 600 → Click Save → toast "Setting updated"
- ☐ Refresh page → value persists as 600
- ☐ Free Delivery Threshold shows 5000 ETB (from seed)
- ☐ Change to 4000 → Save → persists

T8. Delivery Zones (5 min)

- ☐ `/cms/settings/delivery` → Delivery zones page loads
- ☐ Table shows 10 zones (Bole, Kirkos, Arada, ... Lemi Kura)
- ☐ Click "Add Zone" → dialog opens
- ☐ Fill sub-city, base fee → Create → zone appears
- ☐ Edit a zone → pre-filled form → Save
- ☐ Delete a zone → confirm → zone removed

T9. Customer Account Settings (5 min)

- ☐ Log in as customer → Navigate to `/account/settings`
- ☐ Profile form loads with existing data (name, phone, company, address)
- ☐ Change phone number → Save → toast "Profile updated"
- ☐ Refresh → change persists
- ☐ Password change section visible
- ☐ Enter current + new password → Change Password → toast success

T10. Storefront Integrity (5 min)

- ☐ Home page loads without console errors
- ☐ Product detail pages load (no regression from new fields)
- ☐ Cart flow works (add to cart → checkout → order summary)
- ☐ Delivery fee calculation still works (uses hardcoded fallback)
- ☐ Dark mode toggle works across all new pages

---

18. Fix Log (Post-Testing)

#	Area	Issue Found	Severity	Fix Applied	Status

---

19. Documentation Deliverable

After testing is complete, create `docs/v3.6_documentation.md` containing:

1. **Release notes** — What's new in v3.6 (features, fixes, improvements)
2. **Migration guide** — Step-by-step instructions for applying migrations
3. **CMS user guide** — How to use each new CMS feature (categories, products, options, pricing, settings, delivery zones)
4. **API reference** — List of all new API endpoints with request/response formats
5. **Configuration reference** — All site_settings keys and their purpose
6. **Known issues** — Any issues found during testing that are deferred
7. **v3.7 roadmap** — What's next (deferred items from v3.6 scope)

---

20. Execution Order

```
Phase A: User runs migrations on Supabase Dashboard (Section 15)
    │
    ▼
Phase B: I run automated code verification (Section 16)
    │
    ▼
Phase C: User runs manual dev server testing (Section 17)
    │
    ▼
Phase D: Fix any issues found (Section 18)
    │
    ▼
Phase E: Create v3.6 documentation (Section 19)
    │
    ▼
Phase F: Final build verification → DONE
```